

COINCRAFT RESEARCH

Session 1. 선행 과정 점검

선행 커리큘럼과 Kyobo Phase #1의 교집합 분석

2026.05 | M1 S1 | 1시간

COINCRAFT
ACADEMY

이 시간의 목적 — 37시간을 어떻게 활용할 것인가

개요

선행 과정 37시간 커리큘럼과 Kyobo Phase #1 시스템은 목적이 다르다. 겹치는 부분을 정확히 알고 시작해야 이후 내용이 맥락 있게 들린다.

① 쓸 수 있는 것

37시간 중 Phase 1 이해에
직접 활용되는 개념

- 5개 영역
- 자세히 복습한다

② 이 과정에서 안 쓰는 것

Phase 1 범위 밖의 내용
(나쁜 것이 아님)

- Cosmos SDK, DeFi 등
- 빠르게 확인하고 넘긴다

③ 새로 배울 것

선행 과정에서 다루지 않았지만
Phase 1에 필요한 것

- 이 과정의 핵심
- M1~M8에서 다룬다

이 시간이 끝나면: '나는 이미 아는 것과 앞으로 배울 것을 구분할 수 있다'

선행 과정 — 전체 커리큘럼 지도

CH	주제	내용	Phase 1
CH01	블록체인 기초 이해	역사·해시·블록·TX·공개키·합의·NFT 개요·레이어1/2	✓ 일부
CH02	블록체인 지갑	니모닉·HD Wallet·공개키/개인키·실습	✓ 일부
CH03	토큰 이해와 실습	ERC-20·ERC-721·Solidity 기초·Remix 실습	✓ 일부
CH04	합의 알고리즘	BFT/Non-BFT·Probabilistic/Deterministic Finality	✓ 일부
CH05	크로스체인·멀티체인	레이어1/2·브릿지·확장성	✗ 미사용
CH06	스마트컨트랙트·DeFi	Solidity·오라클·DEX·Lending·Staking·NFT+DeFi	✗ 미사용
CH07	블록체인 최신 트렌드	AI+블록체인·RWA·DePIN·میم코인	✗ 미사용
CH08	프로젝트 1: 탈중앙 거래소	Uniswap v2·Web3 프론트엔드·Wagmi·AMM	✗ 미사용
CH09	프로젝트 2: Cosmos SDK	Cosmos SDK·CometBFT·체인 개발	✗ 미사용
CH10	프로젝트 3: Rollkit	셀레스티아·모듈형 블록체인·Rollkit 체인	✗ 미사용

Phase 1 — 당사가 직접 구현하는 것만 본다

당사 직접 구현 (이 과정의 범위)

ⓒ DMZ — 블록체인 인터페이스 미들웨어

- IVASPAAdapter (Strategy Pattern)
- Webhook Receiver (202 패턴)
- Redis Streams + Consumer Group
- DLQ (Dead Letter Queue)
- IBlockchainAdapter (read-only)

㉔ 내부망 — 비즈니스 로직 + 데이터

- 이벤트 조건 판단 (건기 달성 등)
- NFT 발급 요청 생성
- 내부 원장 (TX 상태머신)
- 감사 로그 (SHA-256 체인)
- Admin 대시보드

VASP 위탁 (이 과정 범위 밖)

㉑ BaaS / 지갑·키 관리

- Web3 지갑 SDK
- 사용자 지갑 생성·관리
- TX 서명

㉒ 블록체인 인프라

- 스마트 컨트랙트 배포·운영 (ERC-1155)
- NFT mint/burn/transfer
- 프라이빗 키 보관 (MPC/HSM)
- Node/RPC BaaS
- 온체인 이벤트 → Webhook 발송

→ 교보 담당자는 컨트랙트 코드를 직접 작성·배포하지 않는다

핵심: 스마트 컨트랙트·키 관리·TX 서명은 VASP이 한다. 우리는 비즈니스 로직과 미들웨어를 만든다.

쓸 수 있는 것 — 5개 영역 개요

- 1 해시·블록·TX 구조 → SHA-256 감사 체인 + TX 상태머신 이해
- 2 공개키·개인키·지갑 → 사용자 지갑 주소 매핑 + VASP 키 위탁의 이유
- 3 ERC-20 / ERC-721 → ERC-1155가 왜 필요한지 이해하는 기반
- 4 Finality 개념 (BFT/PoW) → TX 상태머신 CONFIRMED/FINALIZED 설계 근거
- 5 스마트 컨트랙트·이벤트 로그 → IBlockchainAdapter + DMZ 파이프라인 이해

Phase #1 맥락에서 재해석 후 사용

해시 함수 — 선행과정에서 배운 것이 감사 체인에서 쓰인다

선행 과정에서 배운 것

SHA-256: 어떤 입력이든 256비트 고정 출력

특성:

- 결정적: 같은 입력 → 항상 같은 출력
- 단방향: 출력으로 입력 역추적 불가
- 눈사태 효과: 입력 1비트 변경 → 출력 전혀 달라짐
- 충돌 저항: 같은 출력 두 입력 찾기 사실상 불가

Phase 1 감사 체인에서 쓰는 방법

감사로그 N:

$\text{hash} = \text{SHA256}(\text{event} + \text{timestamp} + \text{prev_hash})$

감사로그 N+1:

$\text{prev_hash} = \text{로그N.hash}$

$\text{hash} = \text{SHA256}(\text{event} + \text{timestamp} + \text{prev_hash})$

→ 로그N 변조 → 로그N.hash 변경

→ 로그N+1.prev_hash 불일치

→ 변조 즉시 감지

금감원 요건과의 연결

전자금융거래법: 거래 이력 5년 보관 + 변조 불가 증명 요건

SHA-256 체인 = 온체인 비용 없이 '변조 감지 가능한' DB 원장 구현

→ 선행과정에서 배운 해시 개념이 금융 컴플라이언스 요건 충족에 직접 쓰인다

블록 구조 — parentHash 체인이 내부 원장 설계의 모델

선행 과정에서 배운 블록 구조

블록 헤더:

blockNumber: 19,234,567
parentHash: 이전 블록의 SHA3-256
stateRoot: 전체 계정 상태 머클루트
txRoot: 이 블록 TX들의 머클루트
timestamp: 유닉스 타임스탬프

parentHash = 이전 블록 해시

- 중간 블록 변조 시 이후 모든 해시가 깨짐
- 변조 감지 = 블록체인의 핵심 속성

내부 원장 감사 로그에 같은 원리 적용

내부 DB 감사 로그:

id: 1001
event: NFT_ISSUED
userId: KYB-001
tokenId: 3
txHash: 0xabc...
prev_hash: 로그1000.hash ← parentHash 역할
hash: SHA256(전체 필드)

- 로그1001 변조 → hash 변경
- 로그1002.prev_hash 불일치
- 변조 위치 특정 가능

블록체인의 parentHash 구조를 내부 DB에 복제. 온체인 비용 없이 변조 감지 가능한 감사 원장 구현.

TX 구조 — TX가 무엇인지 알아야 상태머신이 보인다

선행 과정에서 배운 TX

이더리움 TX 구성:

from: 0xSender

to: 0xContract

data: calldata (함수 + 인자)

gasLimit: 최대 소비 허용 가스

maxFeePerGas: EIP-1559

nonce: 발신자 TX 순서 번호

v,r,s: ECDSA 서명

제출 → mempool 대기 → 채굴 → 블록 포함

Phase 1 TX 상태머신 이해

REQUESTED → DB에 기록 (장애 복구 기준점)

SUBMITTED → txHash 획득 (mempool 진입)

PENDING → mempool 대기 (stuck 감지 대상)

MINED → 블록 포함 (1 확인)

CONFIRMED → N 확인 (REORG 위험 없음)

FINALIZED → checkpoint (절대 불변)

→ TX의 생애주기를 알아야
각 상태가 왜 필요한지 이해됨

nonce의 중요성 — PENDING 타임아웃 처리와 직결

같은 nonce로 gasPrice 높은 TX 재제출 = 이전 TX 대체 (gas bump)

다른 nonce로 재제출 = TX 2개가 각각 채굴 = 중복 발행

→ 선행 과정에서 배운 nonce 개념이 PENDING stuck 해결의 핵심

공개키·주소 — 사용자 지갑 주소 매핑의 기반

선행 과정에서 배운 것

개인키(256비트 난수)
→ ECDSA secp256k1
→ 공개키(64바이트)
→ keccak256 해시
→ 마지막 20바이트
→ 이더리움 주소 (0x...)

공개키는 공개해도 됨
개인키로부터 공개키는 계산 가능
공개키로부터 개인키는 계산 불가

Phase 1 사용자 서비스에서 쓰는 방법

사용자 서비스 DB:
userId: KYB-001 (교보 내부 ID)
walletAddr: 0xABCD... (VASP 지갑 주소)
vasp_user_id: WW-9912 (VASP 사용자 ID)

→ NFT 발급 요청 시:
userId → walletAddr 조회
→ mintNFT({ to: walletAddr })

→ 온체인 잔액 조회 시:
walletAddr → getBalance(addr, tokenId)

주소는 공개 정보 — 프라이버시 설계 주의

이더리움 주소(공개키 유도)는 누구나 볼 수 있음 → 온체인에서 해당 주소의 모든 TX 이력 조회 가능
교보 시스템: 주소↔실명 매핑은 내부망에만 보관. 온체인에는 주소만. GDPR/개인정보보호법 대응.

개인키 보관 — 왜 VASP에 위탁하는가

개인키 직접 보관의 문제

선행 과정 실습:

private key = '0xabcd...ef'

→ .env 파일에 저장, 배포 스크립트에서 로드

교보생명이 이렇게 하면:

- ① 서버 침해 시 전체 자산 탈취
→ MINTER_ROLE 키 탈취, 무제한 NFT 발행 가능
- ② 담당자 퇴사 시 키 분실 위험
→ 새 컨트랙트 재배포 필요
- ③ 내부 통제 불가
→ 단독 결정으로 발행, 금감원 내부통제 규정 위반
- ④ 키 백업 체계 부재
→ 재해 복구 불가

VASP HSM 위탁

HSM (Hardware Security Module):

- 키를 하드웨어 칩 안에 격리
- 키 추출 명령 없음
(서명 요청만 가능)
- 물리적 탈취해도 키 접근 불가

VASP MPC (Multi-Party Computation):

- 개인키를 여러 파편으로 분산
- 파편 하나 탈취해도 키 복원 불가
- 서명 시 안전한 다자 계산

교보 시스템:

IVASPAAdapter.mintNFT() 호출

→ VASP API

→ HSM에서 서명

→ 브로드캐스트

→ 교보는 개인키 모름

선행 과정에서 배운 '개인키의 위험성'이 왜 VASP에 키 관리를 위탁하는지 설명한다.

HD Wallet — VASP 사용자별 지갑을 어떻게 생성하는가

선행과정에서 배운 HD Wallet

니모닉 12단어

- 시드(512비트)
- BIP-32 계층 파생
- $m/44'/60'/0'/0/0$ (계정 0)
- $m/44'/60'/0'/0/1$ (계정 1)
- $m/44'/60'/0'/0/N$ (계정 N)

하나의 니모닉으로 무한한 주소 파생
니모닉만 백업하면 모든 계정 복구

VASPi 교보 사용자에게 적용하는 방법

VASP 마스터 니모닉 (HSM 보관)

- $m/44'/60'/\{\text{교보고객ID}\}'/0/0$
- 교보 사용자 A의 지갑 주소

- $m/44'/60'/\{\text{교보고객ID}\}'/0/0$
- 교보 사용자 B의 지갑 주소

- 사용자 100만 명 = 주소 100만 개
- 단일 마스터 니모닉으로 관리
- VASPi 파생 경로로 각 사용자 서명

HD Wallet = '하나의 씨앗에서 무한한 주소' — VASPi 수백만 사용자 지갑을 하나의 HSM으로 관리하는 원리.

ERC-20 — 표준이 왜 필요한지 알아야 ERC-1155가 보인다

선행 과정에서 배운 ERC-20

```
interface IERC20 {  
    totalSupply(): uint256  
    balanceOf(account): uint256  
    transfer(to, amount): bool  
    approve(spender, amount): bool  
    transferFrom(from, to, amount): bool  
}
```

- 이 인터페이스를 구현하면 어떤 DEX·지갑·프로토콜도이 토큰을 처리 가능
- 표준 = 상호운용성

교보 시스템에서 ERC-20이 없는 이유

ERC-20 = 대체 가능 토큰

- 1토큰 = 1토큰 (구분 없음)
- 쿠폰은 종류별 구분이 필요
- '걷기쿠폰'과 '건강검진쿠폰'은 같은 것이 아님

ERC-721 = 대체 불가 (각 토큰 고유)

- 종류별 구분 가능
- 그러나 배치 발행 불가
- 100명에게 발행 = TX 100건

ERC-1155 = 이 두 문제를 동시에 해결

표준의 의미

VASP이 ERC-1155를 구현 → 교보 IBlockchainAdapter는 `getBalance(addr, tokenId)`만 호출하면 됨
표준이 없으면 각 VASP마다 다른 API → 어댑터 구현이 불가능해짐

ERC-721 → ERC-1155 — Kyobo 시스템에서 달라지는 것

ERC-721 (선행 과정 실습)

ownerOf(tokenId) → address

- 토큰 1개의 소유자 1명
- 세상에 하나뿐인 NFT

safeTransferFrom(from, to, tokenId)

- 단건 전송만

교보에 적용 시:

- 쿠폰 종류 × 사용자 수 = 컨트랙트 수
- 100명 발행 = TX 100건

ERC-1155 (이 과정)

balanceOf(account, id) → uint256

- 계정이 tokenId를 몇 개 가졌는가
- 같은 tokenId를 여러 명이 보유

safeBatchTransferFrom(from, to, ids[], amounts[])

- 여러 타입 한 번에 전송

교보에 적용 시:

- 쿠폰 종류 = tokenId로 구분
- 100명 발행 = TX 1건 (mintBatch)

핵심 차이 — balanceOf 시그니처가 달라진다

ERC-721: balanceOf(address owner) → 이 주소가 가진 NFT 총 개수

ERC-1155: balanceOf(address account, uint256 id) → 이 주소가 tokenId=id를 몇 개 보유

→ IBlockchainAdapter.getBalance(addr, operatorAddr, tokenId) 시그니처는 ERC-1155 기반

mintBatch 가스 절감: 단건 × 100 ≈ 5,000,000 gas / mintBatch × 1 ≈ 800,000 gas (84% 절감)

tokenId — ERC-1155에서 가장 중요한 설계 결정

tokenId가 단순 일련번호면

tokenId = 1 → 걷기 달성 쿠폰

tokenId = 2 → 건강검진 쿠폰

tokenId = 3 → 가입기념 쿠폰

...

tokenId = 9999 → ???

문제:

- 숫자만 보면 의미 모름
- 카테고리별 쿼리 불가
(tokenId 1~500이 같은 카테고리인지 모름)
- 스마트컨트랙트 로직에서
타입 검증 불가
- 새 쿠폰 추가 시 관리 테이블 필요

교보 시스템 — 비트 필드 설계

uint256 tokenId 비트 분할:

[255:224] = category

CAT_COUPON = 0x01 << 224

CAT_MEMBERSHIP = 0x02 << 224

CAT_CERT = 0x03 << 224

[223:192] = subcategory

SUB_WALKING = 0x01 << 192

SUB_HEALTH_CHECK = 0x02 << 192

SUB_JOIN_BONUS = 0x03 << 192

예시: 걷기달성 쿠폰 3번째

tokenId = CAT_COUPON | SUB_WALKING | 3

- tokenId만 보고 카테고리 즉시 판별
- bitwise AND로 필터링
- 온체인 추가 저장 불필요



Finality — TX 상태머신 설계의 핵심 근거

선행 과정에서 배운 두 종류 Finality

Probabilistic Finality (PoW/PoS):

- TX가 포함된 블록 위에 블록이 쌓일수록
반복 확률이 지수적으로 감소
- '확정'이 아니라 '확률적으로 안전'
- 이더리움: 32블록 $\approx$ 6분 권장
- REORG 가능성 완전 소멸은 아님

Deterministic Finality (BFT):

- 2/3 이상 검증자 서명 = 즉시 확정
- 단 한 번도 반복 없음
- 코스모스/XRPL: 합의 완료 = 영구 확정
- Finality 시간: 4~6초

Phase 1 TX 상태머신에서의 의미

이더리움 (Probabilistic):

- MINED (1블록): 15초
- REORG 위험 있음 → 쿠폰 사용 불허
- CONFIRMED (32블록): $\sim$ 6분
- 실질적 안전 → 쿠폰 사용 허용
- FINALIZED (checkpoint): $\sim$ 12분
- 절대 불변 → 감사 로그 확정 기록

XRPL (Deterministic BFT):

- CONFIRMED = 합의 완료 즉시
- 4초 후 쿠폰 사용 허용
- FINALIZED = CONFIRMED와 동일

- 어느 체인 쓰느냐에 따라 쿠폰 사용 가능 시점이 달라짐
- IBlockchainAdapter의 존재 이유

Finality 개념을 모르면 왜 CONFIRMED와 FINALIZED를 따로 두는지 이해 안 됨. 선행 과정 합의 수업이 여기서 쓰인다.

REORG — 이것 때문에 CONFIRMED가 필요하다

REORG란

- 두 채굴자가 거의 동시에 블록 발견
- 잠시 체인이 두 갈래로 분기
- 더 긴 체인이 채택
- 짧은 갈래의 블록들은 '무효화'

무효화된 블록의 TX들:

- mempool로 되돌아감
- 또는 영구 유실
- 이미 처리한 것이 사라짐

교보 시스템 REORG 시나리오

- 블록 #100에 NFT mint TX 포함
- Event Consumer가 감지
- 내부 원장: 쿠폰 발행 완료
- 고객 앱: 쿠폰 사용 가능

그런데 블록 #100 REORG 발생

- mint TX 무효화
- 온체인 잔액: 0
- 내부 원장: 1 (불일치)
- 고객이 없는 쿠폰 사용

- MINED 즉시 허용하면 이 일이 생김
- CONFIRMED(32블록) 이후 허용 필요

DLQ에서 REORG 이벤트 처리

VASP Webhook: NFTIssued 이벤트 → Consumer 처리 → 이후 REORG Webhook 수신 → DLQ로 이관 → 운영팀 알림 → 원장 롤백

BFT vs PoW Finality — chainType 설계의 근거

EVM (Probabilistic)
chainType = 'EVM'

Finality: ~12분

TX 상태머신:

MINED → CONFIRMED(32) →
FINALIZED

쿠폰 사용 허용: CONFIRMED
감사 로그 확정: FINALIZED

수수료: gas(wei)
EIP-1559: maxFeePerGas

REORG 대응:
Consumer에서 REORG 이벤트
별도 처리 필요

XRPL (Deterministic BFT)
chainType = 'XRPL'

Finality: ~4초

TX 상태머신:

SUBMITTED → CONFIRMED
(FINALIZED = CONFIRMED)

쿠폰 사용 허용: CONFIRMED
즉시
감사 로그 확정: 동시

수수료: XRP drops (고정)
REORG 없음

→ UX가 완전히 달라짐
버튼 누름 → 4초 → 완료

Circle ARC (BFT)
chainType = 'BFT'

Finality: 즉시

TX 상태머신:

SUBMITTED → FINALIZED
(블록 개념 없음)

쿠폰 사용 허용: API 응답 즉시

수수료: 없음
(Circle 자체 부담)

주의:
탈중앙 보장 없음
Circle 단일 장애점
규제 리스크 별도 검토

선행 과정 합의 알고리즘 수업 → Phase 1 IBlockchainAdapter chainType 리터럴 유니온 설계로 직결된다.

스마트 컨트랙트 — Kyobo 담당자가 코드를 몰라도 알아야 하는 것

선행 과정에서 배운 것 (Remix 실습)

Solidity로 코드 작성

- 컴파일
- 배포 TX → 블록에 포함
- 고유 주소 부여

이후:

- 함수 호출 = TX 발송
- 주소로 calldata 전송
- EVM이 bytecode 실행
- storage 변경
- event 발생

교보 시스템에서 알아야 하는 부분

VASP이 KyoboNFT.sol 배포

- 주소: 0xKyoboNFT

교보 IBlockchainAdapter:

- getBalance(0xKyoboNFT, userAddr, tokenId)
- 해당 주소 컨트랙트에 call 발송 (읽기 전용)

쿠폰 발행:

- mintNFT 호출 → VASP API
- VASP이 TX 서명·발송
- 컨트랙트 내 storage 변경
- NFTIssued event 발생

핵심: Phase 1에서 교보 담당자가 Solidity를 직접 작성하지 않는다

컨트랙트 코드 = VASP 담당. 교보 담당자는 '컨트랙트가 무엇이고 어떻게 호출되는가'를 알면 충분.

- Remix 실습이 준 개념적 이해가 IBlockchainAdapter 설계와 DMZ 이벤트 수신 이해에 쓰인다.

이벤트 로그 — DMZ 파이프라인이 수신하는 것

선행 과정에서 배운 event

Solidity:

```
event Transfer(address indexed from,  
               address indexed to,  
               uint256 tokenId);
```

```
emit Transfer(from, to, tokenId);
```

- TX receipt의 logs 배열에 기록
- 블록에 영구 저장
- 블록체인 노드에서 필터링 가능
- 외부 시스템이 구독 가능

Phase 1 DMZ에서 수신하는 방법

VASP 노드: NFTIssued event 감지

→ Webhook HTTP POST

to: 교보 DMZ Webhook Receiver

body: { event, txHash, tokenId, to, amount }

Webhook Receiver:

→ 즉시 HTTP 202 반환

→ Redis XADD blockchain-events

→ 처리는 Consumer가 비동기로

Consumer:

→ 내부 원장 CONFIRMED 업데이트

→ 멍등키: txHash+logIndex

indexed 파라미터 → 필터링 효율

indexed: 이 파라미터를 topics에 저장 → Bloom Filter로 빠른 검색 가능

non-indexed: data에 저장 → 전체 데이터 파싱 필요

→ Consumer에서 특정 tokenId, 특정 주소의 이벤트만 필터링 시 indexed 활용

contract.on() 한 줄 → 5가지 문제 → DMZ 파이프라인

선행 과정 방식

```
// ethers.js
contract.on('NFTIssued',
  async (to, tokenId, amount) => {
    await db.update(...);
  }
);
```

한 줄로 끝. 직관적이고 간단.

그러나:

- ① 서버 재시작 → 그 사이 이벤트 유실
- ② 순서 보장 없음 (RPC 재전송 시)
- ③ 핸들러 실패 → 이벤트 사라짐
- ④ REORG 이벤트 처리 없음 → 잘못된 이벤트도 처리됨
- ⑤ 핸들러 느리면 전체 블로킹 → 다음 이벤트 밀림

Phase 1 DMZ 파이프라인

- ① 수신 분리:
Webhook Receiver → 202 즉시 반환 → 이벤트 유실 없음
- ② 지속성:
Redis Streams → 재시작 후 재처리
PEL = 처리 중 추적
ACK = 처리 완료 확인
- ③ At-least-once:
ACK 없으면 XAUTOCLAIM으로 다른 Consumer가 재처리
- ④ REORG 대응:
REORG Webhook 별도 처리
원장 롤백 로직
- ⑤ DLQ:
재시도 N회 실패
→ Dead Letter Queue 이관
→ 운영팀 알림 + 수동 처리



contract.on()이 왜 금융 시스템에 안 되는지 이해해야 DMZ 파이프라인의 복잡성이 정당화된다.

이 과정에서 다루지 않는 것 ① — Cosmos SDK · Rollkit

선행 과정 CH08~09에서 배운 것

Cosmos SDK:

- CometBFT 합의 엔진
- 자체 체인 개발 프레임워크
- Bank · Staking 모듈
- IBC (체인 간 통신)

Celestia + Rollkit:

- 모듈형 블록체인
- DA (데이터 가용성) 레이어
- Rollkit으로 체인 배포

→ 자체 블록체인을 만드는 기술

Phase 1에서 안 쓰는 이유

교보 Phase 1 블록체인 선택: 이더리움 (EVM 호환)

이유:

- VASP이 이더리움 EVM 기반
- ERC-1155 표준 지원
- 검증된 보안성 (수년간 운영)
- 금감원 검토 이력 존재

자체 체인을 만드는 것:

- Phase 1 범위 아님
- Cosmos SDK 불필요
- Rollkit 불필요

→ Phase 3 이후 직접 Custody
전환 시에도 EVM 기반 유지 예상

Cosmos SDK와 Rollkit은 뛰어난 기술이다. 하지만 이 시스템은 이더리움 EVM 위에서 작동한다.

이 과정에서 다루지 않는 것 ② — DeFi

선행 과정 CH06에서 배운 것

DeFi 생태계:

- DEX (Uniswap v2 AMM)
- Lending (Aave, Compound)
- Staking / Liquid Staking
- Restaking
- 파생상품 (퍼페추얼, 옵션)
- 스테이블코인
- 에어드랍

Uniswap v2 코드 리뷰:

ERC20.sol · Factory.sol · Pair.sol

교보 시스템이 DeFi가 아닌 이유

DeFi:

사용자가 직접 유동성 공급
AMM 알고리즘으로 가격 결정
스마트컨트랙트가 금융 서비스
무허가 (누구나 접근)

교보 Phase 1:

고객이 걷기 달성 → 쿠폰 수령
쿠폰은 교보 서비스에서만 사용
발행 조건·대상은 내부 시스템 결정
허가형 (교보 고객만)

→ AMM, Lending, Staking 없음

→ Uniswap 코드 불필요

교보 시스템은 'CeFi + 블록체인 기록'에 가깝다. 탈중앙 금융(DeFi) 구조가 아니다.

이 과정에서 다루지 않는 것 ③ — 최신 트렌드

AI + 블록체인

AI agent 온체인
AGI 구현 실험
탈중앙 AI 추론

- Phase 1 무관
- 교보 내부 AI는 별도 시스템

RWA (실물 자산 토큰화)

부동산·채권·금을
블록체인 토큰으로
온보딩

- Phase 1 무관
- 교보 NFT는 RWA 아님
- 금융상품 아님

DePIN

탈중앙 물리 인프라
(와이파이, IoT 등)

- Phase 1 무관
- 교보는 물리 인프라 토큰화 안 함

밈코인

DOGE, PEPE 등
투기성 코인

- Phase 1 무관
- 교보 시스템과 관련 없음

이 내용들은 블록체인 시장 이해에 유용하다. 하지만 Phase 1 구현과는 직접적 관련이 없다.

새로 배울 것들 — 이 과정의 실제 내용

- 1 UUPS Proxy · delegatecall · Storage Collision · AccessControl (M1~M2)
- 2 IVASPAdapter · ExternalIVASPAdapter · Strategy Pattern (M1~M3)
- 3 TX 상태머신 7단계 · requestId Idempotency · gas bump (M3)
- 4 Redis Streams · Consumer Group · PEL · ACK · DLQ (M3~M4)
- 5 IBlockchainAdapter · chainType 리터럴 유니온 · EVMAdapter (M3~M4)
- 6 Gnosis Safe 2-of-3 · EIP-712 SafeTx · execTransaction (M8)
- 7 SHA-256 감사 체인 · append-only 원장 · 감사 검증 알고리즘 (M5~M6)

선행 학습 37시간은 '블록체인이 무엇인가'. 이 과정은 '블록체인을 금융 시스템에 어떻게 연결하는가'.

UUPS Proxy — 선행 과정에서 다루지 않은 영역

※ Universal Upgradeable Proxy Standard

왜 필요한가

컨트랙트는 한번 배포하면 코드를 바꿀 수 없다

그런데:

- 버그 발견 → 수정 필요
- 규제 변경 → 로직 변경
- 기능 추가

→ 새 컨트랙트 재배포하면:
기존 토큰 잔액 migration
고객에게 새 주소 안내
→ 현실적으로 불가능

→ 해결: Proxy 패턴
사용자는 Proxy 주소 사용
실제 로직은 교체 가능

핵심 개념들 (M1~M2에서 상세 학습)

A.delegatecall(0xB의주소):

B의 코드를 A의 몸으로 실행
→ 결과가 A의 storage에 저장

Storage Collision:

Proxy.slot0 = impl 주소
Impl.slot0 = totalSupply
→ delegatecall로 totalSupply 수정
→ impl 주소가 덮어쓰여짐!

ERC-1967:

impl 주소를 특수 슬롯에 저장
→ 충돌 원천 차단

UUPS:

업그레이드 로직을 Impl 안에
→ Proxy 컨트랙트 단순·저렴
→ 업그레이드 = UPGRADER_ROLE 보호

IVASPAdapter — Strategy Pattern으로 VASP를 추상화

왜 추상화가 필요한가

Phase 1: VASP API 연동

Phase 3: 직접 Custody 전환

만약 비즈니스 로직에

VASP API 직접 호출 코드가 있으면:

```
await walletoneApi.mintNFT({...})
```

→ VASP 교체 시

비즈니스 로직 전체 수정

→ 테스트 시 실제 API 호출

→ 모의 VASP로 테스트 불가

→ 해결: Interface로 추상화

IVASPAdapter 설계

```
interface IVASPAdapter {  
  mintNFT(params: MintParams) :  
    Promise<TxRequest>;  
  getWallet(userId: string) : Promise<Wallet>;  
  getTxStatus(txId: string) : Promise<TxStatus>;  
}
```

```
class ExternalIVASPAdapter  
  implements IVASPAdapter {  
  // VASP API 호출  
}
```

```
class MockIVASPAdapter  
  implements IVASPAdapter {  
  // 테스트용 가짜 응답  
}
```

비즈니스 로직:

```
constructor(private vasp: IVASPAdapter)
```

→ VASP 교체 = 생성자 주입만 변경

TX 상태머신 · Idempotency — 분산 시스템의 안전망

TX 상태머신 7단계

REQUESTED

- DB 기록 완료
- 장애 후 재처리 기준점

SUBMITTED

- txHash 획득
- 체인에서 추적 가능

PENDING

- mempool 대기
- stuck 감지 대상 (timeout→gas bump)

MINED → CONFIRMED → FINALIZED

- 각 단계별 비즈니스 정책

FAILED

- DLQ 이관 + 운영팀 알림

Idempotency — 재시도해도 1번만

문제:

- VASP API 호출 → 타임아웃
- 재시도
- VASP가 실은 성공
- 재시도도 성공
- 쿠폰 2개 발행 (이중 지급)

해결:

- 모든 mintNFT 요청에
requestId: UUID 포함

VASP:

1. requestId DB 조회
2. 이미 있으면 기존 결과 반환
3. 없으면 TX 제출 + requestId 저장

- 몇 번 재시도해도 1번만 발행

Redis Streams · DLQ — 이벤트 유실 제로

Redis Streams 핵심 개념

XADD: 메시지 추가

ID = timestamp-sequence (순서 보장)

XREADGROUP: Consumer Group에서 읽기

- PEL(Pending Entry List)에 등록
- 처리 중 추적

XACK: 처리 완료 확인

- PEL에서 제거

XAUTOCLAIM: 일정 시간 ACK 없으면

- 다른 Consumer에 재할당
- At-least-once 보장

XLEN: 스트림 길이 조회

- 적체 모니터링

DLQ (Dead Letter Queue)

재시도 N회 실패 시:

- blockchain-events-dlq 스트림으로 이관
- 원본 이벤트 전체 보존
- 실패 이유 + 스택트레이스
- 재시도 횟수 + 타임스탬프
- 운영팀 알림 발송

원인 수정 후 DLQ 재처리:

- 수동 또는 자동 재투입
- 이벤트 유실 제로

금감원 감사 요건:

- '어떤 이벤트가 처리됐는가'
- 전수 추적 가능해야 함
- DLQ = 이벤트 유실 방지 마지막 안전망

Gnosis Safe — 스마트컨트랙트 지갑과 멀티시그

개인키 MetaMask의 한계

선행 과정 실습:

MetaMask 개인키 하나로
컨트랙트 배포·관리

교보 시스템에서 이렇게 하면:

- 키 1개 탈취 = 전체 권한 탈취
- 단독 결정 → 내부통제 위반
- 금감원 규정: 전자금융거래 시 2인 이상 통제 원칙
- 개인키 분실 = 영구 복구 불가

→ 멀티시그 필요

Gnosis Safe 2-of-3 구성

[K1] 교보생명 보조키 (내부 보관)

[K2] VASP 주키 1 (HSM)

[K3] VASP 주키 2 (별도 HSM)

실행 조건: 3개 중 2개 서명

→ 단독 결정 불가

→ K1 탈취 단독으로는 실행 불가

→ K1 분실해도 K2+K3으로 복구

EIP-712 구조화된 서명:

Kyobo NFT 컨트랙트 업그레이드 승인

to: 0xProxy

data: upgradeToAndCall(newImpl)

→ 서명 내용을 사람이 읽을 수 있음

→ '이게 뭔지 모르고 서명' 방지

M1~M8 학습 로드맵 — 앞으로 배울 것들

M	주제	주요 내용
M1	아키텍처 설계·환경	Phase 1 구조·Hardhat·EVMAdapter·IVASPAAdapter 설계
M2	DMZ 파이프라인	Webhook·202 패턴·Redis Streams·Consumer·DLQ
M3	TX 처리 시스템	TX 상태머신·Idempotency·재시도·gas bump·에러 처리
M4	내부 원장	내부 원장 설계·온체인 reconcile·REORG 처리
M5	감사 로그	SHA-256 체인·append-only·검증 알고리즘·규제 대응
M6	스마트 컨트랙트	KyoboNFT.sol·UUPS Proxy·AccessControl·Pausable
M7	멀티체인 확장	XRPL·Circle ARC·IBlockchainAdapter 확장
M8	거버넌스	Gnosis Safe·EIP-712·SafeTx·UPGRADER_ROLE

Session1 정리 — 지금 내가 어디에 서 있는가

✓ 선행 과정에서 가져오는 것 (5개 영역)

1. 해시·블록·TX 구조
 - SHA-256 감사 체인
 - TX 상태머신 이해
2. 공개키·개인키·지갑
 - 지갑 주소 매핑
 - VASP 키 위탁 이유
3. ERC-20·ERC-721
 - ERC-1155 이해 기반
4. Finality 개념
 - TX 상태 설계 근거
 - chainType 차이
5. 스마트 컨트랙트·이벤트 로그
 - IBlockchainAdapter
 - DMZ 파이프라인

✗ 이 과정에서 다루지 않는 것

- Cosmos SDK
 - Rollkit · 셀레스티아
 - Uniswap · AMM · DeFi
 - AI+블록체인
 - RWA · DePIN
 - 밌코인
 - 크로스체인 브릿지
- 잘못 배운 것이 아님
→ Phase 1 범위 밖
→ 다른 시스템·프로젝트에서 분명히 쓸 기회가 있음

NEW 새로 배울 것 (이 과정의 핵심)

- UUPS Proxy
delegatecall · ERC-1967
- IVASPAAdapter
Strategy Pattern
- TX 상태머신
Idempotency · gas bump
- Redis Streams
Consumer Group · DLQ
- Gnosis Safe
EIP-712 · SafeTx
- SHA-256 감사 체인
변조 감지 · 잔액 재현
- TypeScript 타입 시스템
리터럴 유니온 · 인터페이스